

Chapter 25 — Graph DFS

0. What You Will Learn in This Chapter

Graph DFS is one of the most important graph traversal patterns.

DFS means:

Depth-First Search

Graph DFS explores as far as possible along one path before backtracking.

It is used for:

- reachability
- connected components
- cycle detection
- path search
- graph cloning
- island counting
- grid traversal
- directed graph traversal
- recursion over graph structures
- backtracking over graph paths

By the end of this chapter, you should understand:

- What Graph DFS is
- How Graph DFS differs from Tree DFS
- How to use a visited set
- How to traverse undirected graphs
- How to traverse directed graphs
- How to handle disconnected graphs
- How to count connected components
- How to detect cycles
- How to solve island/grid problems
- How to use DFS with string-labeled graphs
- How recursive DFS works
- How iterative DFS works with stack
- How to avoid infinite loops
- How to reason about DFS complexity

Core problems covered in this chapter:

1. Find if Path Exists in Graph
2. Number of Connected Components in an Undirected Graph
3. Number of Provinces
4. Keys and Rooms
5. Clone Graph
6. All Paths From Source to Target
7. Number of Islands
8. Max Area of Island
9. Flood Fill
10. Surrounded Regions
11. Pacific Atlantic Water Flow
12. Reorder Routes to Make All Paths Lead to City Zero
13. Evaluate Division
14. Accounts Merge

- 15. Course Schedule, DFS cycle detection
 - 16. Redundant Connection, contrast with Union Find
 - 17. Critical Connections, advanced bridge DFS preview
-

1. What Is Graph DFS?

Graph DFS is a traversal method that explores deeply before exploring other branches.

Given graph:

```
0 -- 1 -- 3
|
2
```

DFS from 0 might visit:

```
0 -> 1 -> 3 -> 2
```

or:

```
0 -> 2 -> 1 -> 3
```

depending on neighbor order.

The exact order is usually not important.

The important idea is:

```
go deep, mark visited, backtrack
```

2. Why Graph DFS Exists

Graphs are not linear.

From one node, you may have many possible neighbors.

DFS gives a systematic way to explore all reachable nodes.

Example:

```
void dfs(int node) {
    visited[node] = true;

    for (int neighbor : graph[node]) {
        if (!visited[neighbor]) {
            dfs(neighbor);
        }
    }
}
```

This visits every node reachable from node.

3. Graph DFS vs Tree DFS

Tree DFS and Graph DFS are similar, but graphs have a major extra issue:

cycles

In a tree, there is exactly one path from root to each node.

In a graph, you can return to a previously visited node.

Example:

0 -- 1

| |

2 -- 3

If DFS does not track visited nodes, it may loop forever:

0 -> 1 -> 0 -> 1 -> ...

So graph DFS almost always needs:

visited

4. Core Need / Why Use Graph DFS?

Use DFS when you need to explore all reachable states from a starting point.

DFS is especially natural for:

- connected components
- reachability
- recursive exploration
- grid islands
- path generation
- cycle detection
- graph coloring
- flood fill
- dependency cycle detection

5. When to Use Graph DFS

Use Graph DFS when the problem asks:

5.1 Can One Node Reach Another?

Examples:

- Find if Path Exists in Graph
- Keys and Rooms
- Evaluate Division

5.2 Count Connected Components

Examples:

- Number of Connected Components
- Number of Provinces
- Accounts Merge

5.3 Explore Grid Regions

Examples:

- Number of Islands

- Max Area of Island
- Flood Fill
- Surrounded Regions

5.4 Generate All Paths

Examples:

- All Paths From Source to Target

5.5 Detect Cycles

Examples:

- Course Schedule
- Graph Valid Tree
- Redundant Connection concept

5.6 Mark Reachability

Examples:

- Pacific Atlantic Water Flow
 - Surrounded Regions
-

6. When NOT to Use Graph DFS

6.1 Need Shortest Path in Unweighted Graph

Use BFS.

DFS may find a path, but not necessarily shortest.

Examples:

- Shortest Path in Binary Matrix
- Word Ladder
- Open the Lock

6.2 Need Weighted Shortest Path

Use Dijkstra, Bellman-Ford, or Floyd-Warshall.

6.3 Need Dynamic Connectivity

Use Union Find.

6.4 Need Topological Order

DFS can do topological sort, but Kahn's BFS indegree algorithm is often easier.

6.5 Graph Depth Can Be Huge

Recursive DFS may cause stack overflow.

Use iterative DFS or BFS.

7. Core Intuition

DFS answers:

If I start here and keep going as deep as possible, what can I reach?

The three most important pieces are:

18. graph representation
19. visited tracking
20. recursive or stack traversal

Basic DFS:

```
visited[node] = true;
```

```
for each neighbor:
```

```
    if not visited:
```

```
        dfs(neighbor)
```

8. Mental Models

8.1 Maze Exploration

DFS walks down a path until it cannot continue, then backs up and tries another path.

8.2 Paint Bucket

Starting from one node, DFS spreads to all connected nodes and marks them. This is useful for islands and connected components.

8.3 Call Stack as Path

Recursive DFS uses the call stack to remember where to return after finishing a branch.

8.4 Visited Means Already Processed

Once a node is visited, do not process it again.

This prevents infinite loops and duplicate work.

9. Pattern Recognition Checklist

Ask:

21. Is this a graph or grid?
22. Do I need to visit all reachable nodes?
23. Could cycles exist?
24. Do I need a visited array/set?
25. Is the graph disconnected?
26. Should I start DFS from every unvisited node?
27. Is this a connected component problem?
28. Is this a region-filling problem?
29. Is this path generation rather than shortest path?
30. Is recursion depth safe?

If yes, Graph DFS may apply.

10. C++ Tools Needed

10.1 Visited Vector for Integer Nodes

```
vector<bool> visited(n, false);
```

10.2 Visited Set for String Nodes

```
unordered_set<string> visited;
```

10.3 Recursive Function

```
void dfs(int node, vector<vector<int>>& graph, vector<bool>& visited)
```

10.4 Iterative Stack

```
stack<int> st;
```

10.5 Grid Directions

```
vector<pair<int, int>> dirs = {  
    {1, 0},  
    {-1, 0},  
    {0, 1},  
    {0, -1}  
};
```

11. Core Templates

Template 1 — Recursive DFS, Integer Graph

```
void dfs(int node, vector<vector<int>>& graph, vector<bool>& visited) {  
    visited[node] = true;  
  
    for (int neighbor : graph[node]) {  
        if (!visited[neighbor]) {  
            dfs(neighbor, graph, visited);  
        }  
    }  
}
```

Template 2 — Iterative DFS With Stack

```
void dfsIterative(int start, vector<vector<int>>& graph, vector<bool>& visited) {  
    stack<int> st;  
    st.push(start);  
    visited[start] = true;  
  
    while (!st.empty()) {  
        int node = st.top();  
        st.pop();  
  
        for (int neighbor : graph[node]) {  
            if (!visited[neighbor]) {  
                visited[neighbor] = true;  
                st.push(neighbor);  
            }  
        }  
    }  
}
```

```
    }  
  }  
}
```

Template 3 — Count Connected Components

```
int countComponents(int n, vector<vector<int>>& graph) {  
    vector<bool> visited(n, false);  
    int components = 0;  
  
    for (int i = 0; i < n; i++) {  
        if (!visited[i]) {  
            components++;  
            dfs(i, graph, visited);  
        }  
    }  
  
    return components;  
}
```

Template 4 — DFS Path Exists

```
bool dfs(int node, int target,  
        vector<vector<int>>& graph,  
        vector<bool>& visited) {  
    if (node == target) {  
        return true;  
    }  
  
    visited[node] = true;  
  
    for (int neighbor : graph[node]) {  
        if (!visited[neighbor]) {  
            if (dfs(neighbor, target, graph, visited)) {  
                return true;  
            }  
        }  
    }  
  
    return false;  
}
```

Template 5 — Grid DFS

```
void dfs(vector<vector<char>>& grid, int r, int c) {
    int rows = grid.size();
    int cols = grid[0].size();

    if (r < 0 || c < 0 || r >= rows || c >= cols) {
        return;
    }

    if (grid[r][c] != '1') {
        return;
    }

    grid[r][c] = '0';

    dfs(grid, r + 1, c);
    dfs(grid, r - 1, c);
    dfs(grid, r, c + 1);
    dfs(grid, r, c - 1);
}
```

Template 6 — DFS With Path Backtracking

```
void dfs(int node,
        int target,
        vector<vector<int>>& graph,
        vector<int>& path,
        vector<vector<int>>& result) {
    if (node == target) {
        result.push_back(path);
        return;
    }

    for (int neighbor : graph[node]) {
        path.push_back(neighbor);

        dfs(neighbor, target, graph, path, result);

        path.pop_back();
    }
}
```

Used for DAG path generation.

Template 7 — Directed Graph Cycle Detection

```
bool dfs(int node, vector<vector<int>>& graph, vector<int>& state) {
    if (state[node] == 1) {
        return true;
    }

    if (state[node] == 2) {
        return false;
    }

    state[node] = 1;

    for (int neighbor : graph[node]) {
        if (dfs(neighbor, graph, state)) {
            return true;
        }
    }

    state[node] = 2;

    return false;
}
```

State meanings:

0 = unvisited
1 = visiting, currently in recursion stack
2 = done

12. Major DFS Variations

Reachability DFS	can source reach target	Valid Path
Component DFS	start from every unvisited node	Connected Components
Grid DFS	flood-fill neighboring cells	Number of Islands
Path DFS	push/pop current path	All Paths DAG
Clone DFS	map old node to new node	Clone Graph

Variation	Core Idea	Example
Directed cycle DFS	3-color state	Course Schedule
Undirected cycle DFS	track parent	Graph Valid Tree
Reverse reachability	DFS from boundaries	Pacific Atlantic
Weighted DFS	carry product/cost	Evaluate Division

13. Full Problem Walkthrough 1 — LeetCode 1971: Find if Path Exists in Graph

13.1 Problem Summary

Given undirected graph with n nodes, edges, source, and destination.
Return true if source can reach destination.

13.2 How to Identify DFS

Need reachability.
Use DFS from source.
If destination is visited, return true.

13.3 C++ Code

```
class Solution {
public:
    bool dfs(int node,
             int destination,
             vector<vector<int>>& graph,
             vector<bool>& visited) {
        if (node == destination) {
            return true;
        }

        visited[node] = true;

        for (int neighbor : graph[node]) {
            if (!visited[neighbor]) {
                if (dfs(neighbor, destination, graph, visited)) {
                    return true;
                }
            }
        }
    }
};
```

```

    }
}

return false;
}

bool validPath(int n,
               vector<vector<int>>& edges,
               int source,
               int destination) {
    vector<vector<int>> graph(n);

    for (auto& edge : edges) {
        int u = edge[0];
        int v = edge[1];

        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    vector<bool> visited(n, false);

    return dfs(source, destination, graph, visited);
}
};

```

13.4 Complexity

Build graph:

$O(n + m)$

DFS:

$O(n + m)$

Space:

$O(n + m)$

13.5 Common Mistake

For an undirected graph, forgetting to add both directions.

14. Full Problem Walkthrough 2 — Number of Connected Components

14.1 Problem Summary

Given n nodes and undirected edges, return number of connected components.

Example:

```
n = 5  
edges = [[0,1], [1,2], [3,4]]
```

Components:

```
{0,1,2}  
{3,4}
```

Answer:

```
2
```

14.2 How to Identify DFS

Need count disconnected groups.

Run DFS from every unvisited node.

Each new DFS means a new component.

14.3 C++ Code

```
class Solution {  
public:  
    void dfs(int node,  
             vector<vector<int>>& graph,  
             vector<bool>& visited) {  
        visited[node] = true;  
  
        for (int neighbor : graph[node]) {  
            if (!visited[neighbor]) {  
                dfs(neighbor, graph, visited);  
            }  
        }  
    }  
  
    int countComponents(int n, vector<vector<int>>& edges) {  
        vector<vector<int>> graph(n);  
  
        for (auto& edge : edges) {  
            int u = edge[0];  
            int v = edge[1];  
  
            graph[u].push_back(v);  
            graph[v].push_back(u);  
        }  
    }  
};
```

```
vector<bool> visited(n, false);
int components = 0;

for (int node = 0; node < n; node++) {
    if (!visited[node]) {
        components++;
        dfs(node, graph, visited);
    }
}

return components;
};
```

14.4 Complexity

Time:

$O(n + m)$

Space:

$O(n + m)$

15. Full Problem Walkthrough 3 — LeetCode 547: Number of Provinces

15.1 Problem Summary

Given matrix `isConnected`.

`isConnected[i][j] = 1` means city `i` and city `j` are directly connected.

Return number of provinces.

A province is a connected component.

15.2 Representation Choice

Input is adjacency matrix.

Can DFS directly on matrix.

No need to build adjacency list.

15.3 C++ Code

```
class Solution {
public:
```

```

void dfs(int city,
        vector<vector<int>>& isConnected,
        vector<bool>& visited) {
    visited[city] = true;

    int n = isConnected.size();

    for (int next = 0; next < n; next++) {
        if (isConnected[city][next] == 1 && !visited[next]) {
            dfs(next, isConnected, visited);
        }
    }
}

int findCircleNum(vector<vector<int>>& isConnected) {
    int n = isConnected.size();

    vector<bool> visited(n, false);
    int provinces = 0;

    for (int city = 0; city < n; city++) {
        if (!visited[city]) {
            provinces++;
            dfs(city, isConnected, visited);
        }
    }

    return provinces;
}
};

```

15.4 Complexity

Time:

$O(n^2)$

because matrix neighbor scan checks all cities.

Space:

$O(n)$

recursion and visited.

15.5 Common Mistake

Trying to treat matrix rows as sparse adjacency lists.

Matrix DFS scans all possible neighbors.

16. Full Problem Walkthrough 4 — LeetCode 841: Keys and Rooms

16.1 Problem Summary

You start in room 0.

Each room contains keys to other rooms.

Return true if all rooms can be visited.

16.2 How to Identify DFS

Rooms are nodes.

Keys are directed edges.

Need reachability from room 0.

16.3 C++ Code

```
class Solution {
public:
    void dfs(int room,
             vector<vector<int>>& rooms,
             vector<bool>& visited) {
        visited[room] = true;

        for (int key : rooms[room]) {
            if (!visited[key]) {
                dfs(key, rooms, visited);
            }
        }
    }

    bool canVisitAllRooms(vector<vector<int>>& rooms) {
        int n = rooms.size();

        vector<bool> visited(n, false);

        dfs(0, rooms, visited);

        for (bool seen : visited) {
            if (!seen) {
                return false;
            }
        }
    }
};
```

```
        return true;
    }
};
```

16.4 Complexity

Time:

$O(n + \text{totalKeys})$

Space:

$O(n)$

17. Full Problem Walkthrough 5 — LeetCode 133: Clone Graph

17.1 Problem Summary

Given a reference node in a connected undirected graph, return a deep copy.
Each node has:

```
int val;
vector<Node*> neighbors;
```

17.2 How to Identify DFS

Need visit every reachable original node and create corresponding clone.
Because graph may have cycles, use map:

original node -> cloned node

17.3 C++ Code

```
class Solution {
public:
    unordered_map<Node*, Node*> copied;

    Node* cloneGraph(Node* node) {
        if (node == nullptr) {
            return nullptr;
        }

        if (copied.count(node)) {
            return copied[node];
        }
    }
};
```

```

Node* clone = new Node(node->val);
copied[node] = clone;

for (Node* neighbor : node->neighbors) {
    clone->neighbors.push_back(cloneGraph(neighbor));
}

return clone;
}
};

```

17.4 Complexity

Time:

$O(V + E)$

Space:

$O(V)$

17.5 Common Mistakes

- Not using map and cloning nodes repeatedly
- Infinite recursion on cycles
- Returning original node instead of clone
- Copying only values, not edges

18. Full Problem Walkthrough 6 — LeetCode 797: All Paths From Source to Target

18.1 Problem Summary

Given DAG with nodes $0..n-1$, return all paths from 0 to $n-1$.

18.2 How to Identify DFS

Need generate all paths.

This is DFS with backtracking.

Because graph is DAG, no visited set is necessary for cycle prevention.

18.3 C++ Code

```

class Solution {
public:
    void dfs(vector<vector<int>>& graph,
            int node,

```

```

        vector<int>& path,
        vector<vector<int>>& result) {
    int target = graph.size() - 1;

    if (node == target) {
        result.push_back(path);
        return;
    }

    for (int neighbor : graph[node]) {
        path.push_back(neighbor);

        dfs(graph, neighbor, path, result);

        path.pop_back();
    }
}

vector<vector<int>> allPathsSourceTarget(vector<vector<int>>& graph) {
    vector<vector<int>> result;
    vector<int> path;

    path.push_back(0);

    dfs(graph, 0, path, result);

    return result;
}
};

```

18.4 Complexity

Number of paths can be exponential.

Time:

$O(\text{total output size})$

Space:

$O(n)$

for current path, excluding output.

18.5 Common Mistake

Using a global visited set can wrongly block valid paths in a DAG when different paths share nodes.

For all paths in DAG, use path backtracking, not global visited.

19. Full Problem Walkthrough 7 — LeetCode 200: Number of Islands

19.1 Problem Summary

Given grid of:

'1' = land

'0' = water

Return number of islands.

Island is connected land horizontally or vertically.

19.2 How to Identify DFS

Grid cells are graph nodes.

Land connected to land forms components.

Count connected components of land.

19.3 C++ Code

```
class Solution {
public:
    void dfs(vector<vector<char>>& grid, int r, int c) {
        int rows = grid.size();
        int cols = grid[0].size();

        if (r < 0 || c < 0 || r >= rows || c >= cols) {
            return;
        }

        if (grid[r][c] != '1') {
            return;
        }

        grid[r][c] = '0';

        dfs(grid, r + 1, c);
        dfs(grid, r - 1, c);
        dfs(grid, r, c + 1);
        dfs(grid, r, c - 1);
    }

    int numIslands(vector<vector<char>>& grid) {
        int rows = grid.size();
```

```
int cols = grid[0].size();

int islands = 0;

for (int r = 0; r < rows; r++) {
    for (int c = 0; c < cols; c++) {
        if (grid[r][c] == '1') {
            islands++;
            dfs(grid, r, c);
        }
    }
}

return islands;
}
```

19.4 Complexity

Time:

$O(\text{rows} * \text{cols})$

Space:

$O(\text{rows} * \text{cols})$

worst-case recursion stack.

19.5 Common Mistakes

- Not marking visited
- Counting diagonals when only 4-direction allowed
- Forgetting boundary checks
- Using integer 1 instead of character '1'

20. Full Problem Walkthrough 8 — LeetCode 695: Max Area of Island

20.1 Problem Summary

Given grid of:

1 = land
0 = water

Return largest island area.

20.2 How to Identify DFS

Need size of each connected land component.
DFS returns area.

20.3 C++ Code

```
class Solution {
public:
    int dfs(vector<vector<int>>& grid, int r, int c) {
        int rows = grid.size();
        int cols = grid[0].size();

        if (r < 0 || c < 0 || r >= rows || c >= cols) {
            return 0;
        }

        if (grid[r][c] == 0) {
            return 0;
        }

        grid[r][c] = 0;

        return 1 +
            dfs(grid, r + 1, c) +
            dfs(grid, r - 1, c) +
            dfs(grid, r, c + 1) +
            dfs(grid, r, c - 1);
    }

    int maxAreaOfIsland(vector<vector<int>>& grid) {
        int rows = grid.size();
        int cols = grid[0].size();

        int best = 0;

        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < cols; c++) {
                if (grid[r][c] == 1) {
                    best = max(best, dfs(grid, r, c));
                }
            }
        }

        return best;
    }
};
```

```
}  
};
```

20.4 Complexity

Time:

$O(\text{rows} * \text{cols})$

Space:

$O(\text{rows} * \text{cols})$

worst-case recursion stack.

21. Full Problem Walkthrough 9 — LeetCode 733: Flood Fill

21.1 Problem Summary

Given image matrix, starting cell, and new color.

Change connected region of original color to new color.

21.2 How to Identify DFS

This is classic flood fill.

Starting from one cell, spread to neighbors with same original color.

21.3 C++ Code

```
class Solution {  
public:  
    void dfs(vector<vector<int>>& image,  
            int r,  
            int c,  
            int originalColor,  
            int newColor) {  
        int rows = image.size();  
        int cols = image[0].size();  
  
        if (r < 0 || c < 0 || r >= rows || c >= cols) {  
            return;  
        }  
  
        if (image[r][c] != originalColor) {  
            return;  
        }  
    }  
};
```

```

        image[r][c] = newColor;

        dfs(image, r + 1, c, originalColor, newColor);
        dfs(image, r - 1, c, originalColor, newColor);
        dfs(image, r, c + 1, originalColor, newColor);
        dfs(image, r, c - 1, originalColor, newColor);
    }

    vector<vector<int>> floodFill(vector<vector<int>>& image,
                                int sr,
                                int sc,
                                int color) {
        int originalColor = image[sr][sc];

        if (originalColor == color) {
            return image;
        }

        dfs(image, sr, sc, originalColor, color);

        return image;
    }
};

```

21.4 Why Check originalColor == color?

If new color equals original color, DFS would keep revisiting the same cells forever unless separately visited.

Return early.

21.5 Complexity

Time:

$O(\text{rows} * \text{cols})$

Space:

$O(\text{rows} * \text{cols})$

worst-case recursion stack.

22. Full Problem Walkthrough 10 — LeetCode 130: Surrounded Regions

22.1 Problem Summary

Given board with:

'X'
'O'

Capture all regions of O completely surrounded by X.
Any O connected to border should not be captured.

22.2 How to Identify DFS

Instead of finding surrounded regions directly, find safe regions.

Any O connected to border is safe.

DFS from border Os and mark them.

Then flip remaining O to X.

22.3 C++ Code

```
class Solution {
public:
    void dfs(vector<vector<char>>& board, int r, int c) {
        int rows = board.size();
        int cols = board[0].size();

        if (r < 0 || c < 0 || r >= rows || c >= cols) {
            return;
        }

        if (board[r][c] != 'O') {
            return;
        }

        board[r][c] = '#';

        dfs(board, r + 1, c);
        dfs(board, r - 1, c);
        dfs(board, r, c + 1);
        dfs(board, r, c - 1);
    }

    void solve(vector<vector<char>>& board) {
        int rows = board.size();
        int cols = board[0].size();

        for (int r = 0; r < rows; r++) {
            dfs(board, r, 0);
        }
    }
};
```

```

        dfs(board, r, cols - 1);
    }

    for (int c = 0; c < cols; c++) {
        dfs(board, 0, c);
        dfs(board, rows - 1, c);
    }

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (board[r][c] == 'O') {
                board[r][c] = 'X';
            } else if (board[r][c] == '#') {
                board[r][c] = 'O';
            }
        }
    }
}
};

```

22.4 Complexity

Time:

$O(\text{rows} * \text{cols})$

Space:

$O(\text{rows} * \text{cols})$

worst-case recursion stack.

22.5 Common Mistake

Trying to DFS every O and decide if it touches border is more complicated.
Start from border safe cells instead.

23. Full Problem Walkthrough 11 — LeetCode 417: Pacific Atlantic Water Flow

23.1 Problem Summary

Given heights grid.

Water can flow from cell to neighbor if neighbor height is less than or equal to current height.

Return cells that can flow to both Pacific and Atlantic oceans.

Pacific touches top and left borders.
Atlantic touches bottom and right borders.

23.2 How to Identify DFS

Directly simulating from every cell is expensive.

Reverse thinking:

Start DFS from oceans and move uphill.

A cell can reach an ocean if ocean DFS can reach that cell by moving to greater/equal heights.

23.3 C++ Code

```
class Solution {
public:
    vector<pair<int, int>> dirs = {
        {1, 0},
        {-1, 0},
        {0, 1},
        {0, -1}
    };

    void dfs(vector<vector<int>>& heights,
            int r,
            int c,
            vector<vector<bool>>& visited) {
        int rows = heights.size();
        int cols = heights[0].size();

        visited[r][c] = true;

        for (auto [dr, dc] : dirs) {
            int nr = r + dr;
            int nc = c + dc;

            if (nr < 0 || nc < 0 || nr >= rows || nc >= cols) {
                continue;
            }

            if (visited[nr][nc]) {
                continue;
            }

            if (heights[nr][nc] < heights[r][c]) {
                continue;
            }
        }
    }
};
```

```

    }

    dfs(heights, nr, nc, visited);
}
}

vector<vector<int>> pacificAtlantic(vector<vector<int>>& heights) {
    int rows = heights.size();
    int cols = heights[0].size();

    vector<vector<bool>> pacific(rows, vector<bool>(cols, false));
    vector<vector<bool>> atlantic(rows, vector<bool>(cols, false));

    for (int r = 0; r < rows; r++) {
        dfs(heights, r, 0, pacific);
        dfs(heights, r, cols - 1, atlantic);
    }

    for (int c = 0; c < cols; c++) {
        dfs(heights, 0, c, pacific);
        dfs(heights, rows - 1, c, atlantic);
    }

    vector<vector<int>> result;

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (pacific[r][c] && atlantic[r][c]) {
                result.push_back({r, c});
            }
        }
    }

    return result;
}
};

```

23.4 Complexity

Time:

$O(\text{rows} * \text{cols})$

Each ocean DFS visits each cell at most once.

Space:

$O(\text{rows} * \text{cols})$

23.5 Common Mistake

Flow direction is reversed in DFS.

From ocean, move to neighbor if:

neighbor height $\geq$ current height

not lower.

24. Full Problem Walkthrough 12 — LeetCode 1466: Reorder Routes to Make All Paths Lead to City Zero

24.1 Problem Summary

There are n cities and directed roads.

Need minimum number of edges to reverse so every city can reach city 0.

24.2 How to Identify DFS

Tree-like connected graph.

Need traverse from city 0 and count edges originally going away from 0.

Representation:

For original edge:

$u \rightarrow v$

Store:

$u \rightarrow v$ cost 1

$v \rightarrow u$ cost 0

24.3 C++ Code

```
class Solution {
public:
    int dfs(int node,
            vector<vector<pair<int, int>>>& graph,
            vector<bool>& visited) {
        visited[node] = true;

        int changes = 0;

        for (auto [neighbor, cost] : graph[node]) {
            if (!visited[neighbor]) {
```

```

        changes += cost;
        changes += dfs(neighbor, graph, visited);
    }
}

return changes;
}

int minReorder(int n, vector<vector<int>>& connections) {
    vector<vector<pair<int, int>>> graph(n);

    for (auto& edge : connections) {
        int u = edge[0];
        int v = edge[1];

        graph[u].push_back({v, 1});
        graph[v].push_back({u, 0});
    }

    vector<bool> visited(n, false);

    return dfs(0, graph, visited);
}
};

```

24.4 Complexity

Time:

$O(n)$

because there are $n - 1$ roads.

Space:

$O(n)$

25. Full Problem Walkthrough 13 — LeetCode 399: Evaluate Division

25.1 Problem Summary

Given equations:

$a / b = \text{value}$

Answer queries:

x / y

If not possible, return -1.0.

25.2 How to Identify DFS

Variables are graph nodes.

Equation creates weighted edges:

$a \rightarrow b = \text{value}$

$b \rightarrow a = 1 / \text{value}$

Query asks for product along path from numerator to denominator.

25.3 C++ Code

```
class Solution {
public:
    bool dfs(string current,
             string target,
             unordered_map<string, vector<pair<string, double>>>& graph,
             unordered_set<string>& visited,
             double product,
             double& answer) {
        if (current == target) {
            answer = product;
            return true;
        }

        visited.insert(current);

        for (auto [neighbor, weight] : graph[current]) {
            if (!visited.count(neighbor)) {
                if (dfs(neighbor, target, graph, visited,
                        product * weight, answer)) {
                    return true;
                }
            }
        }

        return false;
    }

    vector<double> calcEquation(vector<vector<string>>& equations,
                               vector<double>& values,
                               vector<vector<string>>& queries) {
```

```

unordered_map<string, vector<pair<string, double>>> graph;

for (int i = 0; i < equations.size(); i++) {
    string a = equations[i][0];
    string b = equations[i][1];
    double value = values[i];

    graph[a].push_back({b, value});
    graph[b].push_back({a, 1.0 / value});
}

vector<double> result;

for (auto& query : queries) {
    string start = query[0];
    string target = query[1];

    if (!graph.count(start) || !graph.count(target)) {
        result.push_back(-1.0);
        continue;
    }

    unordered_set<string> visited;
    double answer = -1.0;

    if (dfs(start, target, graph, visited, 1.0, answer)) {
        result.push_back(answer);
    } else {
        result.push_back(-1.0);
    }
}

return result;
}
};

```

25.4 Complexity

Let:

V = number of variables
 E = number of equations
 Q = number of queries

Each query DFS:

$O(V + E)$

Total:

$O(Q * (V + E))$

25.5 Common Mistake

For reciprocal edge, use:

1.0 / value

not:

-value

26. Full Problem Walkthrough 14 — Accounts Merge

26.1 Problem Summary

Accounts with common email belong to same person.
Merge emails and return sorted emails for each person.

26.2 How to Identify DFS

Emails are graph nodes.
Emails in same account are connected.
Connected component of emails belongs to same person.

26.3 C++ Code

```
class Solution {
public:
    void dfs(string email,
             unordered_map<string, vector<string>>& graph,
             unordered_set<string>& visited,
             vector<string>& component) {
        visited.insert(email);
        component.push_back(email);

        for (string& neighbor : graph[email]) {
            if (!visited.count(neighbor)) {
                dfs(neighbor, graph, visited, component);
            }
        }
    }
}
```

```

vector<vector<string>> accountsMerge(vector<vector<string>>& accounts) {
    unordered_map<string, vector<string>> graph;
    unordered_map<string, string> emailToName;

    for (auto& account : accounts) {
        string name = account[0];
        string firstEmail = account[1];

        for (int i = 1; i < account.size(); i++) {
            string email = account[i];

            emailToName[email] = name;

            graph[firstEmail].push_back(email);
            graph[email].push_back(firstEmail);
        }
    }

    unordered_set<string> visited;
    vector<vector<string>> result;

    for (auto& [email, name] : emailToName) {
        if (visited.count(email)) {
            continue;
        }

        vector<string> component;
        dfs(email, graph, visited, component);

        sort(component.begin(), component.end());

        vector<string> merged;
        merged.push_back(emailToName[email]);

        for (string& e : component) {
            merged.push_back(e);
        }

        result.push_back(merged);
    }

    return result;
}

```

```
}  
};
```

26.4 Complexity

Let E be total number of emails across all accounts.

Building graph:

$O(E)$

DFS:

$O(E)$

Sorting components:

$O(E \log E)$

Overall:

$O(E \log E)$

26.5 Common Mistake

Using account index as node can work, but email-as-node is often simpler because shared email defines connection.

27. Full Problem Walkthrough 15 — LeetCode 207: Course Schedule, DFS Cycle Detection

27.1 Problem Summary

There are `numCourses` courses and prerequisites.

Each pair:

`[a, b]`

means take `b` before `a`.

Return true if all courses can be finished.

27.2 How to Identify DFS Cycle Detection

Courses form directed graph.

If there is a cycle, impossible.

Use DFS with 3 states:

0 = unvisited

1 = visiting

2 = done

If DFS reaches a visiting node, cycle exists.

27.3 C++ Code

```
class Solution {
public:
    bool hasCycle(int course,
                  vector<vector<int>>& graph,
                  vector<int>& state) {
        if (state[course] == 1) {
            return true;
        }

        if (state[course] == 2) {
            return false;
        }

        state[course] = 1;

        for (int next : graph[course]) {
            if (hasCycle(next, graph, state)) {
                return true;
            }
        }

        state[course] = 2;

        return false;
    }

    bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
        vector<vector<int>> graph(numCourses);

        for (auto& pre : prerequisites) {
            int course = pre[0];
            int prereq = pre[1];

            graph[prereq].push_back(course);
        }

        vector<int> state(numCourses, 0);

        for (int course = 0; course < numCourses; course++) {
            if (state[course] == 0) {
                if (hasCycle(course, graph, state)) {
```

```

        return false;
    }
}

return true;
}
};

```

27.4 Complexity

Time:

$O(V + E)$

Space:

$O(V + E)$

27.5 Common Mistakes

- Using only visited for directed cycle detection
- Not distinguishing currently visiting from already completed
- Building prerequisite edge in wrong direction

28. Full Problem Walkthrough 16 — Undirected Graph Cycle Detection

28.1 Problem Summary

Given undirected graph, determine if it contains a cycle.

28.2 Key Idea

In undirected graph, seeing a visited neighbor is not always a cycle because it may be the parent.

Track parent.

If neighbor is visited and neighbor is not parent, cycle exists.

28.3 C++ Code

```

bool hasCycleUndirected(int node,
    int parent,
    vector<vector<int>>& graph,
    vector<bool>& visited) {
    visited[node] = true;

    for (int neighbor : graph[node]) {
        if (!visited[neighbor]) {

```

```

        if (hasCycleUndirected(neighbor, node, graph, visited)) {
            return true;
        }
    } else if (neighbor != parent) {
        return true;
    }
}

return false;
}

```

28.4 Important Difference

Directed graph cycle detection needs recursion stack state.
 Undirected graph cycle detection needs parent tracking.

29. Mini Problem 1 — DFS Count Reachable Nodes

```

int dfsCount(int node, vector<vector<int>>& graph, vector<bool>& visited) {
    visited[node] = true;

    int count = 1;

    for (int neighbor : graph[node]) {
        if (!visited[neighbor]) {
            count += dfsCount(neighbor, graph, visited);
        }
    }

    return count;
}

```

30. Mini Problem 2 — Iterative DFS for Grid

```

void dfsGridIterative(vector<vector<int>>& grid, int sr, int sc) {
    int rows = grid.size();
    int cols = grid[0].size();
}

```

```

stack<pair<int, int>> st;
st.push({sr, sc});

grid[sr][sc] = 0;

vector<pair<int, int>> dirs = {
    {1, 0},
    {-1, 0},
    {0, 1},
    {0, -1}
};

while (!st.empty()) {
    auto [r, c] = st.top();
    st.pop();

    for (auto [dr, dc] : dirs) {
        int nr = r + dr;
        int nc = c + dc;

        if (nr < 0 || nc < 0 || nr >= rows || nc >= cols) {
            continue;
        }

        if (grid[nr][nc] == 0) {
            continue;
        }

        grid[nr][nc] = 0;
        st.push({nr, nc});
    }
}
}

```

31. Mini Problem 3 — DFS With Parent Tracking

```

void dfs(int node,
        int parent,
        vector<vector<int>>& graph,
        vector<bool>& visited) {

```

```
visited[node] = true;

for (int neighbor : graph[node]) {
    if (neighbor == parent) {
        continue;
    }

    if (!visited[neighbor]) {
        dfs(neighbor, node, graph, visited);
    }
}
}
```

32. Mini Problem 4 — DFS Component Collection

```
void dfsCollect(int node,
               vector<vector<int>>& graph,
               vector<bool>& visited,
               vector<int>& component) {
    visited[node] = true;
    component.push_back(node);

    for (int neighbor : graph[node]) {
        if (!visited[neighbor]) {
            dfsCollect(neighbor, graph, visited, component);
        }
    }
}
```

33. Common Edge Cases for Graph DFS

33.1 Empty Graph

If $n = 0$, handle according to problem.

33.2 Single Node

Reachability from node to itself is true.

33.3 Disconnected Graph

If problem asks about all nodes, start DFS from every unvisited node.

33.4 Cycles

Always use visited or state tracking.

33.5 Self Loop

A self-loop can be a cycle.

33.6 Duplicate Edges

DFS still works if visited is used.

33.7 Grid Boundary

Always check row and column bounds.

33.8 Recursion Depth

Large graph or grid can cause stack overflow.

Use iterative DFS if needed.

34. Common Mistakes and Wrong Code

Mistake 1 — No Visited Set in Graph

Wrong:

```
for (int neighbor : graph[node]) {  
    dfs(neighbor);  
}
```

Can loop forever.

Correct:

```
if (!visited[neighbor]) {  
    dfs(neighbor);  
}
```

Mistake 2 — Marking Visited Too Late

Wrong:

```
for neighbor:  
    dfs(neighbor)  
visited[node] = true;
```

Correct:

```
visited[node] = true;  
for neighbor:  
    ...
```

Mark immediately when entering or before pushing into stack.

Mistake 3 — Only Starting DFS From Node 0

For disconnected graphs, this misses components.

Correct:

```
for (int i = 0; i < n; i++) {  
    if (!visited[i]) {  
        dfs(i);  
    }  
}
```

Mistake 4 — Wrong Directed Cycle Detection

A normal visited array cannot distinguish:

- current recursion path
- already completed safe node

Use 3-state DFS.

Mistake 5 — Not Backtracking Path

For all paths:

```
path.push_back(x);  
dfs(...);  
path.pop_back();
```

Mistake 6 — Mutating Grid When Original Must Be Preserved

If problem needs original grid later, use separate visited matrix instead of changing cells.

35. Interview Explanation Templates

35.1 Reachability

I build an adjacency list and run DFS from the source. If I reach the destination, I return true. I mark nodes visited to avoid revisiting nodes in cycles.

35.2 Connected Components

I loop through all nodes. Whenever I find an unvisited node, that starts a new component, and DFS marks every node in that component.

35.3 Number of Islands

I treat each land cell as a graph node. When I find unvisited land, I count a new island and use DFS to mark the entire island as visited.

35.4 Clone Graph

I use a hash map from original nodes to cloned nodes. DFS clones each node once and recursively clones its neighbors.

35.5 Directed Cycle Detection

I use three states: unvisited, visiting, and done. If DFS reaches a visiting node, there is a cycle.

36. Complexity Patterns

Graph reachability	$O(V + E)$	$O(V + E)$
Connected components	$O(V + E)$	$O(V + E)$
Number of Provinces matrix	$O(n^2)$	$O(n)$
Keys and Rooms	$O(V + E)$	$O(V)$
Clone Graph	$O(V + E)$	$O(V)$
All Paths in DAG	exponential output	$O(\text{path length})$
Number of Islands	$O(R * C)$	$O(R * C)$
Max Area of Island	$O(R * C)$	$O(R * C)$
Flood Fill	$O(R * C)$	$O(R * C)$
Surrounded Regions	$O(R * C)$	$O(R * C)$
Pacific Atlantic	$O(R * C)$	$O(R * C)$
Evaluate Division	$O(Q * (V + E))$	$O(V + E)$
Course Schedule DFS	$O(V + E)$	$O(V + E)$

37. Time-Boxed Practice Guidance

Easy Graph DFS

Target:

15–25 minutes

Examples:

- Find if Path Exists
- Keys and Rooms
- Number of Provinces
- Flood Fill

Medium Graph DFS

Target:

25–45 minutes

Examples:

- Number of Islands
- Max Area of Island
- Clone Graph
- Accounts Merge

- Course Schedule DFS

Harder Graph DFS

Target:

45–75 minutes

Examples:

- Pacific Atlantic Water Flow
 - Surrounded Regions
 - Evaluate Division
 - Critical Connections, advanced
-

38. Expanded Practice Set

38.1 Must-Do Easy

31. Find if Path Exists in Graph
32. Keys and Rooms
33. Number of Provinces
34. Flood Fill

38.2 Must-Do Medium

35. Number of Connected Components
36. Number of Islands
37. Max Area of Island
38. Clone Graph
39. All Paths From Source to Target
40. Surrounded Regions
41. Pacific Atlantic Water Flow
42. Reorder Routes to Make All Paths Lead to City Zero
43. Evaluate Division
44. Accounts Merge
45. Course Schedule

38.3 Strong Interview Practice

46. Course Schedule II, DFS topological sort
47. Graph Valid Tree
48. Redundant Connection
49. Number of Distinct Islands
50. Making A Large Island
51. Eventual Safe States
52. Find Eventual Safe States
53. Critical Connections in a Network

38.4 Hard / Stretch

54. Critical Connections in a Network
 55. Word Ladder II, BFS + DFS
 56. Remove Invalid Parentheses, graph/search concept
 57. Longest Increasing Path in a Matrix, DFS + memo
 58. Alien Dictionary, graph + topo
 59. Minimum Height Trees, graph trimming
 60. Strongly Connected Components, advanced
-

39. Suggested Study Order

Step 1 — Basic Reachability

Start with:

- 61. Find if Path Exists
- 62. Keys and Rooms
- 63. Number of Provinces

Goal:

Learn visited DFS.

Step 2 — Connected Components

Solve:

- 64. Number of Connected Components
- 65. Accounts Merge

Goal:

Learn starting DFS from every unvisited node.

Step 3 — Grid DFS

Solve:

- 66. Flood Fill
- 67. Number of Islands
- 68. Max Area of Island
- 69. Surrounded Regions

Goal:

Treat grids as graphs.

Step 4 — Path and Clone DFS

Solve:

- 70. Clone Graph
- 71. All Paths From Source to Target
- 72. Evaluate Division

Goal:

Learn DFS with maps, paths, and weights.

Step 5 — Cycle Detection

Solve:

- 73. Course Schedule
- 74. Graph Valid Tree
- 75. Eventual Safe States

Goal:

Learn DFS state tracking.

Step 6 — Advanced Reachability

Solve:

- 76. Pacific Atlantic Water Flow
- 77. Critical Connections

Goal:

40. Pattern Decision Table

source can reach target	DFS from source	Valid Path
count groups	DFS every unvisited	Components
matrix of connections	DFS over matrix row	Provinces
rooms and keys	directed DFS	Keys and Rooms
clone graph	DFS + old-to-new map	Clone Graph
all paths	DFS + path backtracking	All Paths DAG
islands	grid DFS mark visited	Number of Islands
largest region	DFS returns size	Max Area Island
recolor region	flood-fill DFS	Flood Fill
border-safe regions	DFS from border	Surrounded Regions
ocean reachability	reverse DFS from boundaries	Pacific Atlantic
weighted equation path	DFS with product	Evaluate Division
directed cycle	3-state DFS	Course Schedule
undirected cycle	DFS with parent	Graph Valid Tree

41. Final Cheat Sheet

Basic DFS

```
void dfs(int node) {
    visited[node] = true;

    for (int neighbor : graph[node]) {
        if (!visited[neighbor]) {
            dfs(neighbor);
        }
    }
}
```

```
    }  
}
```

Component Count

```
for (int i = 0; i < n; i++) {  
    if (!visited[i]) {  
        components++;  
        dfs(i);  
    }  
}
```

Grid DFS

```
if (r < 0 || c < 0 || r >= rows || c >= cols) {  
    return;  
}  
  
if (grid[r][c] != target) {  
    return;  
}  
  
grid[r][c] = visitedMarker;  
  
dfs(r + 1, c);  
dfs(r - 1, c);  
dfs(r, c + 1);  
dfs(r, c - 1);
```

Directed Cycle Detection

```
if (state[node] == 1) {  
    return true;  
}  
  
if (state[node] == 2) {  
    return false;  
}  
  
state[node] = 1;  
  
for (int neighbor : graph[node]) {  
    if (dfs(neighbor)) {  
        return true;  
    }  
}
```

```
state[node] = 2;
```

```
return false;
```

Path Backtracking

```
path.push_back(neighbor);
```

```
dfs(neighbor);
```

```
path.pop_back();
```

Iterative DFS

```
stack<int> st;
```

```
st.push(start);
```

```
visited[start] = true;
```

```
while (!st.empty()) {
```

```
    int node = st.top();
```

```
    st.pop();
```

```
    for (int neighbor : graph[node]) {
```

```
        if (!visited[neighbor]) {
```

```
            visited[neighbor] = true;
```

```
            st.push(neighbor);
```

```
        }
```

```
    }
```

```
}
```

42. What to Master Before Chapter 26

Before moving to Graph BFS, make sure you can:

- Build adjacency list
- Run recursive DFS
- Run iterative DFS
- Use visited array
- Handle disconnected graphs
- Count connected components
- Solve reachability
- Solve Number of Islands
- Solve Max Area of Island
- Use grid directions

- Clone graph with hash map
 - Generate all paths with backtracking
 - Detect directed cycle with 3 states
 - Detect undirected cycle with parent
 - Avoid infinite recursion
 - Explain DFS complexity as $O(V + E)$
-

43. Core Takeaway

Graph DFS explores deeply through a graph while marking visited nodes.
The most important rule is:

Mark nodes as visited so cycles do not cause infinite traversal.

Graph DFS is best for:

- reachability
- connected components
- grid regions
- recursive graph exploration
- path generation
- cycle detection

The key question is:

From this node, what connected nodes can I fully explore?

If the task is about discovering everything reachable from a start point or grouping connected regions, DFS is often the simplest solution.
